DIM0501 — Boas Práticas de Programação

Projeto da Unidade 1 — AgendaFácil (versão reduzida)

Individual ou em equipe de até 5 — consolidação das Aulas 1 a 5 · Versão Python

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Entrega: 25/10/2026

Integrantes (nome e matrícula)	Linguagem escolhida

O que é este trabalho

Você recebe um sistema que funciona mal por dentro e o coloca em ordem sem quebrá-lo. Não há funcionalidade nova a implementar: todo o trabalho é sobre a qualidade interna do que já existe — organização e modularização (Aula 3), documentação (Aula 4) e programação defensiva (Aula 5), partindo de um diagnóstico feito com o checklist da Aula 2.

> **Sobre o tamanho da equipe:** o escopo é o mesmo para uma pessoa ou para cinco. Em equipe, a divisão natural é por etapa — mas todos respondem pelo arquivo inteiro.

O sistema AgendaFácil

O AgendaFácil é uma agenda de atendimentos de uma clínica pequena. Ele marca um atendimento, remarca e lista o que está agendado. O código foi escrito por estagiários que já saíram da empresa, nunca passou por revisão e está inteiro em um único arquivo.

Código-base fornecido

```
# ============================================================
# Arquivo: agenda_base.py
# Autor: Equipe de estagiarios - 03/02/2024
# Alterado por: Joao - 15/05/2025
# Alterado por: Marina - 09/01/2026
# ============================================================

ag = []


def add(p, pr, d, h, dur):
    # adiciona na lista
    try:
        if h >= 8:
            if h < 18:
                if dur > 0:
                    ok = True
                    for a in ag:
                        if a[2] == d and a[3] == h and a[1] == pr:
                            ok = False
                    if ok:
                        ag.append([p, pr, d, h, dur, "MARCADO"])
                        print("Atendimento marcado para", p)
                    else:
                        print("Horario ocupado")
    except:
        pass


def rem(p, d, h, nd, nh):
    # remarca o atendimento
    for a in ag:
        if a[0] == p and a[2] == d and a[3] == h:
            ok = True
            for b in ag:
                if b[2] == nd and b[3] == nh and b[1] == a[1]:
                    ok = False
            if ok:
                a[2] = nd
                a[3] = nh
                print("Remarcado")
            else:
                print("Horario ocupado")


# def desmarcar(p, d, h):
#     ag.remove(...)

# TODO: rever isso depois (Joao, 2025)


def lst(d):
    # lista os atendimentos do mes
    for a in ag:
        if a[2] == d:
            print(a[3], a[0], a[1], a[5])


add("Ana", "Dr. Silva", "2026-10-05", 9, 30)
add("Bruno", "Dr. Silva", "2026-10-05", 9, 30)
add("Carla", "Dra. Souza", "2026-10-05", 14, 60)
rem("Ana", "2026-10-05", 9, "2026-10-06", 10)
lst("2026-10-05")
```

Arquivo: agenda_base.py · Para executar: python3 agenda_base.py

> **Saída atual do programa:** Atendimento marcado para Ana / Horario ocupado / Atendimento marcado para Carla / Remarcado / 14 Carla Dra. Souza MARCADO

> **Dica de leitura:** Uma dataclass ou uma classe simples para o atendimento resolve de uma vez o problema das listas acessadas por índice.

Etapa 1 — Diagnóstico

Antes de tocar em qualquer linha, produza o diagnóstico do código-base usando as cinco categorias do checklist da Aula 2. Para cada anomalia, registre onde ela está e por que é um problema. Mínimo de dez anomalias no total, com pelo menos uma em cada categoria.

Categoria	Anomalia encontrada (e onde)	Por que é um problema
1. Adequação e lógica		
2. Manutenibilidade		
3. Organização e estética		
4. Programação defensiva e segurança		
5. Desempenho e recursos		

Esta tabela vai para o RELATORIO e é a primeira seção avaliada. Ela também é a sua linha de base: ao final, você deve conseguir dizer o que resolveu e o que decidiu deixar como dívida técnica assumida.

Etapa 2 — Refatoração

Reorganize o código preservando exatamente o comportamento atual para as entradas do programa de demonstração.

→ nenhuma função com mais de uma responsabilidade — quem valida não agenda, quem agenda não imprime;

→ a verificação de conflito de horário deve existir em um único lugar;

→ nenhum valor fixo com significado de negócio solto no código;

→ o atendimento deve virar um tipo com campos nomeados, em vez de uma estrutura acessada por posição;

→ os nomes seguem a convenção da linguagem: PEP 8 — snake_case para funções e variáveis, PascalCase para classes, MAIUSCULAS_COM_UNDERSCORE para constantes.

> **Regra de ouro:** ao final desta etapa, o programa deve imprimir exatamente a mesma saída de hoje. Refatorar é mudar a estrutura sem mudar o comportamento.

Etapa 3 — Documentação

→ toda função pública recebe o contrato completo em docstrings no estilo Google (Sphinx), com as seções Args, Returns e Raises;

→ os comentários de ruído, o cabeçalho com histórico de alterações, o código morto comentado e o TODO vencido devem desaparecer;

→ escreva pelo menos um comentário de decisão, explicando um porquê que o código não revela;

→ o README deve permitir que alguém de fora compile, execute e gere a documentação sem perguntar nada;

→ gere a documentação de API com pdoc ou sphinx-apidoc.

Etapa 4 — Programação defensiva

O sistema passa a tratar toda entrada como hostil.

→ as pré-condições são verificadas no início de cada função pública, com cláusulas de guarda;

→ entradas inválidas geram exceção específica (ValueError, ou uma exceção própria que herde de Exception) com mensagem que diz o que está errado e qual valor causou o problema;

→ nenhum bloco de captura vazio ou genérico pode permanecer no código;

→ nenhuma operação pode falhar em silêncio: hoje, marcar um atendimento às 20h simplesmente não faz nada, e ninguém fica sabendo.

> **Atenção:** aqui o comportamento para entradas inválidas muda de propósito, e essa mudança precisa estar registrada no RELATORIO. Só o caminho feliz permanece idêntico.

Regras de negócio a validar

Algumas já existem no código-base, outras não — descobrir quais são quais faz parte do diagnóstico.

Regra	Definição
Expediente	Atendimentos só podem começar entre 8h e 17h; o último termina até as 18h.
Duração	Entre 15 e 120 minutos, em múltiplos de 15.
Conflito	Um profissional não pode ter dois atendimentos no mesmo dia e horário.
Data	Formato AAAA-MM-DD. Qualquer outro formato é entrada inválida.

Restrições técnicas

→ uma única linguagem, entre Python, Java e C++ (mínimo: Python 3.10 ou superior);

→ apenas a biblioteca padrão — nenhuma dependência externa;

→ sem banco de dados e sem interface gráfica: os dados vivem em memória;

→ o programa de demonstração do código-base deve continuar existindo e rodando.

Entregáveis

- ✦ **Código refatorado —** Na linguagem escolhida, compilando e executando com o mesmo programa de demonstração do código-base.
- ✦ **README.md —** Curto: o que é o sistema, como compilar e executar, como gerar a documentação e quem fez.
- ✦ **RELATORIO.md (ou PDF) —** Também curto: a tabela de diagnóstico da Etapa 1 e um parágrafo por decisão de projeto relevante.
- ✦ **Pasta docs/ —** A documentação de API gerada pela ferramenta da linguagem — ou o comando que a gera, documentado no README.

Entrega até 25/10/2026, às 23h59, em repositório Git com acesso ao professor ou em arquivo compactado único.

Critérios de avaliação

Critério	Pontos
Diagnóstico — anomalias identificadas, classificadas e justificadas	2,0
Organização — uma responsabilidade por função, sem duplicação, sem valores fixos soltos	2,5
Nomenclatura — nomes descritivos e convenção da linguagem respeitada	1,5
Documentação — contrato das funções públicas, README e comentários de decisão	2,0
Programação defensiva — validação, guardas, exceções específicas, nenhum erro engolido	2,0
Total	**10,0**

Penalidades e integridade

- ✕ atraso: menos 1,0 ponto por dia corrido, até cinco dias; depois disso, a entrega não é aceita;
- ✕ alteração do comportamento do caminho feliz sem justificativa no RELATORIO: menos 1,0 ponto;
- ✕ uso de biblioteca externa: a parte correspondente é desconsiderada;
- ✕ o uso de ferramentas de IA é permitido e deve ser declarado no RELATORIO. O professor pode pedir que qualquer integrante explique qualquer trecho entregue — não saber explicar é tratado como não ter escrito.

Código que funciona mas não se explica não está pronto.